

Streaming in LangGraph

How to replace the blocking `.invoke()` call with `.stream()` so your chatbot delivers a real, token-by-token typewriter experience — in both the LangGraph backend and the Streamlit UI.

WHAT THIS CHAPTER COVERS

- ▶ What streaming means in LLM applications
- ▶ Why streaming matters — four concrete reasons
- ▶ `.invoke()` vs `.stream()` — the core difference
- ▶ Python generators as the mechanism behind `.stream()`
- ▶ `stream_mode="messages"` and its purpose
- ▶ Reading `(message_chunk, metadata)` tuples
- ▶ Implementing streaming in the LangGraph backend
- ▶ Implementing streaming in Streamlit with `st.write_stream`

Streaming in LangGraph

RECAP — CHAPTER 11

The previous chapter gave the LangGraph chatbot a Streamlit UI: a persistent `message_history` list in `st.session_state`, a `st.chat_input` box, and `st.chat_message` containers rendering each turn. Every response was fetched with a single blocking `chatbot.invoke(...)` call and displayed only once the entire answer was ready. This chapter fixes that blocking behavior by introducing streaming.

1. The Problem With `.invoke()` on Long Responses

The Symptom

When the existing chatbot (built with `chatbot.invoke(...)`) was asked a long-output question — for example, *"Write a 500-word blog on cricket"* — the UI froze. The screen stayed completely blank for several seconds, and then the entire 500-word response appeared all at once.

This happens because the LLM must finish generating the **full** response internally before LangGraph's `.invoke()` call returns anything to the caller. The UI has nothing to render until that single return value is available.

Why This Is a Real Problem

1. **Perceived latency:** the user waits 5–10 seconds (or longer, depending on output length) staring at a blank or frozen-looking screen.
2. **Poor readability:** a large block of text appearing instantly is harder to read and process than the same text appearing progressively.

`.invoke()` — BLOCKING (WHOLE RESPONSE AT ONCE)



`.stream()` — TOKEN-BY-TOKEN (TYPEWRITER EFFECT)



Same final content, two very different user experiences — wait-then-dump versus continuous delivery.

Figure 12.1 — `.invoke()` forces the user to wait for the entire response before anything appears; `.stream()` delivers each token the moment it is generated.

IMPORTANT

Streaming does not change *what* the LLM generates or how long generation takes internally. It changes *when and how* the generated content is delivered to the caller — token by token as it becomes available, instead of as one final blob.

Definition

In LLMs, *streaming* means the model starts sending tokens as soon as they are generated, instead of waiting for the entire response to be ready before returning it. This produces the familiar "typewriter effect" seen in tools such as ChatGPT, where words appear to be typed out in front of the user rather than dumped on screen all at once.

🔄 Quick Revision — The Problem With `.invoke()`

- `chatbot.invoke(...)` blocks until the LLM has generated the complete response.
- For long outputs (e.g. a 500-word blog), this creates a multi-second blank-screen wait.
- Streaming = sending tokens as soon as each one is generated, instead of waiting for the full response.
- The result is a token-by-token, typewriter-style UI experience.

2. Why Streaming Matters

Streaming is a small implementation change with an outsized impact on user experience. There are four main reasons it is considered a near-mandatory feature for any serious LLM-based application.

1 Faster Perceived Response

User sees output immediately instead of waiting 5–10s for long answers

2 Feels Human & Engaging

Mimics natural conversation, builds trust, keeps the user engaged

3 Better Readability for Long Output

Code and long text are easier to follow when printed step-by-step

4 Early Stop Saves Tokens & Cost

User can interrupt mid-response; unneeded tokens are never generated

Figure 12.2 — The four core reasons streaming is worth implementing in any LLM-based product.

1. FASTER PERCEIVED RESPONSE TIME

Even though the LLM takes the same total time to finish generating, the user sees the first tokens almost immediately. This drastically reduces the feeling of "nothing is happening," which matters especially for users who are not deeply technical — a frozen-looking screen can make them assume the app has crashed and abandon it.

2. MIMICS HUMAN-LIKE CONVERSATION

Streaming makes the interaction feel alive and builds trust, because it resembles how a person actually types or speaks — progressively, not all at once. This is also directly relevant to multimodal or voice interfaces (e.g. an Alexa-like device):

without streaming, the device would have to wait for the entire response before it could even begin speaking, breaking the natural rhythm of conversation.

3. BETTER READABILITY FOR LONG OUTPUT, ESPECIALLY CODE

When an LLM generates code and the entire block appears instantly, it is genuinely harder to follow — the reader cannot easily tell where one part ends and the next begins. Watching the code appear line-by-line makes the structure much easier to track.

4. EARLY STOPPING SAVES TOKENS (AND MONEY)

If a user does not like the direction a response is taking, streaming lets them interrupt it mid-generation. Because most LLM providers charge based on the number of tokens generated, stopping early means fewer tokens are produced and billed.

REAL WORLD INSIGHT

Streaming is not limited to displaying an LLM's final message. Agentic applications also stream intermediate **status updates** — for example, an AI agent booking a movie ticket can stream updates like "Opening BookMyShow" → "Selecting movie" → "Selecting seat" → "Selecting payment mode" → "Processing payment," rather than leaving the user uncertain for a full minute before a single final confirmation appears.

🔄 Quick Revision — Why Streaming Matters

- Faster perceived response time reduces user drop-off, especially for non-technical users.
- Streaming mimics natural human conversation and keeps the user engaged.
- Long output (especially code) is far more readable when it appears progressively.
- Early stopping mid-response saves generated tokens, which directly saves cost.
- Agentic apps also stream step-by-step status updates, not just the final LLM message.

3. Generators: The Mechanism Behind `.stream()`

Definition

In Python, a *generator* is a special type of iterator that allows values to be produced on the fly, one at a time, using the `yield` keyword instead of `return`. Rather than computing an entire result set and returning it at once, a generator function pauses after each `yield` and resumes only when the next value is requested.

How It Connects to LangGraph

Instead of calling `graph.invoke(...)` to execute a compiled graph, LangGraph provides `graph.stream(...)`. Calling `.stream()` does not return a final answer directly — it returns a **generator object**. Looping over that generator is what actually drives execution and yields output incrementally.

```
# Confirming the return type of .stream()
print(type(stream))
# output: <class 'generator'>
```

Because a generator is an iterator, the natural way to consume its output is to loop over it:

```
for message_chunk, metadata in chatbot.stream(
    {'messages': [HumanMessage(content='Tell me the recipe to make pasta.')]},
    config={'configurable': {'thread_id': 'thread_1'}},
    stream_mode='messages'
):
    if message_chunk.content:
        print(message_chunk.content, end="", flush=True)
```

COMMON MISTAKE

Treating `.stream()` like `.invoke()` and expecting a single return value is a common early mistake. `.stream()` never returns a finished answer directly — it returns a generator that must be iterated over (typically with a `for` loop) to actually receive output.

🔄 Quick Revision — Generators

- A Python generator produces values one at a time using `yield`, instead of returning everything at once.
- `graph.stream(...)` returns a generator object, not a final answer.
- `type(stream)` confirms this: it prints `<class 'generator'>`.
- You must loop over the generator (a `for` loop) to actually receive streamed output.

4. `stream_mode` and the `(message_chunk, metadata)` Tuple

stream_mode Options

LangGraph's `.stream()` accepts a `stream_mode` argument that controls what kind of information is streamed back. The main modes available are:

stream_mode	What It Streams
messages	LLM output tokens, one chunk at a time — used for token-by-token typewriter display.
updates	State updates produced by each node as the graph executes.
values	The full state value after each step of graph execution.
custom	Arbitrary custom events emitted explicitly from within a node (useful for tool/agent status updates).

For this chapter's goal — streaming an LLM's response token by token — `stream_mode='messages'` is the only mode needed. The remaining modes become relevant later, when building agentic applications that use tools and need to surface step-by-step status updates.

Reading the Streamed Output: (`message_chunk`, `metadata`)

When `stream_mode='messages'` is used, each item yielded by the generator is a tuple of two elements:

- **`message_chunk`** — the actual piece of the LLM's output for this step, accessed via `message_chunk.content`.
- **`metadata`** — additional information about that chunk (e.g. which node produced it).

This is why the loop is written as `for message_chunk, metadata in stream:` — unpacking both elements of each yielded tuple. Only `message_chunk.content` is needed to print the growing response; `metadata` is not used for this simple use case but will matter in more advanced streaming scenarios (e.g. distinguishing which node/agent a chunk came from).

INTERVIEW TIP

If asked to explain LangGraph streaming modes, be precise: `stream_mode='messages'` streams LLM token chunks, while `'updates'` and `'values'` stream graph-execution state rather than raw LLM text. Confusing these is a common way to lose marks in an interview setting.

🔄 Quick Revision — `stream_mode`

- `stream_mode='messages'` streams LLM output tokens one chunk at a time.
- Other modes (`updates`, `values`, `custom`) stream graph state/events, used later for agentic status updates.
- Each streamed item with `stream_mode='messages'` is a (`message_chunk`, `metadata`) tuple.
- `message_chunk.content` holds the actual text; check it exists before printing/using it.

5. Implementing Streaming in the LangGraph Backend

The existing backend (`langgraph_backend.py`) already defines the compiled graph as `chatbot`, with a single `chat_node` that calls the LLM and an `InMemorySaver` checkpointer. No changes are required to the graph definition itself — the graph is built exactly the same way whether it is later invoked or streamed.

The Existing Graph (Unchanged)

```
from langchain_core.messages import BaseMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver
from langchain_groq import ChatGroq
from typing import TypedDict, Annotated
from dotenv import load_dotenv

load_dotenv()

model = ChatGroq(model = "openai/gpt-oss-20b")

# Define State
class MessageState(TypedDict):
    messages : Annotated[List[BaseMessage], add_messages]

# Define Chatbot function
def chat_node(state : MessageState):
    messages = state['messages']

    response = model.invoke(messages).content

    return {'messages':[response]}

# Define Graph
graph = StateGraph(MessageState)

# Nodes & Edges
graph.add_node('chat_node', chat_node)
graph.add_edge(START, 'chat_node')
graph.add_edge('chat_node', END)

# Checkpointer
checkpointer = InMemorySaver()

chatbot = graph.compile(checkpointer = checkpointer)
```

COMMON MISTAKE

Note that inside `chat_node`, the response is captured with `model.invoke(messages).content` — a normal blocking call at the individual node level. This is correct and unrelated to graph-level streaming: `chat_node` still runs to completion internally on every graph step. What changes for streaming is only how the **graph itself** is executed from the outside — via `chatbot.stream(...)` instead of `chatbot.invoke(...)` — not how any individual node computes its result.

The Only Backend Change: `.invoke()` → `.stream()`

Previously, the graph was executed with a single blocking call:

```
response = chatbot.invoke(
    {'messages': [HumanMessage(content='What is the recipe to make pasta?')]}},
    config={'configurable': {'thread_id': 'thread_1'}}
)
```

To enable streaming, this is replaced with `.stream()`, adding the `stream_mode` argument and looping over the returned generator:

```
# Returns a generator object
stream = chatbot.stream(
    {'messages': [HumanMessage(
        content='Tell me the recipe to make a totally homemade pizza.')]},
    config = {'configurable': {'thread_id': 'thread_1'}},
    stream_mode = 'messages'
)

for message_chunk, metadata in stream:
    if message_chunk.content:
        print(message_chunk.content, end = "", flush = True)
```

Running this prints the response token by token in the terminal, rather than all at once — confirming the mechanism works at the backend level before wiring it into any UI.

BEST PRACTICE

Always guard with `if message_chunk.content:` before printing or forwarding a chunk. Some yielded chunks may carry empty or non-text content (e.g. metadata-only steps), and printing them unconditionally can introduce blank output or errors downstream.

🔄 Quick Revision — Backend Implementation

- The `StateGraph` definition, nodes, edges, and checkpointers remain completely unchanged for streaming.
- Only the execution call changes: `chatbot.invoke(...)` becomes `chatbot.stream(...)`.
- `chatbot.stream(...)` needs the initial state, config (with `thread_id`), and `stream_mode='messages'`.
- Loop over the returned generator with: `for message_chunk, metadata in stream:`
- Print/forward only when `message_chunk.content` is truthy.

6. Implementing Streaming in the Streamlit Frontend

The UI Element: `st.write_stream`

Streamlit ships several chat-specific UI elements. This project has already used two of them — `st.chat_input` and `st.chat_message`. Two more are relevant to agentic and streaming UIs:

- **`st.status`** — a container for showing step-by-step status updates for agent-style applications (e.g. "Getting data", "Searching", "Found URL").
- **`st.write_stream`** — renders generators and streams to the app with a typewriter effect. This is the element needed here: pass it a generator, and it handles the entire progressive-rendering UI automatically.

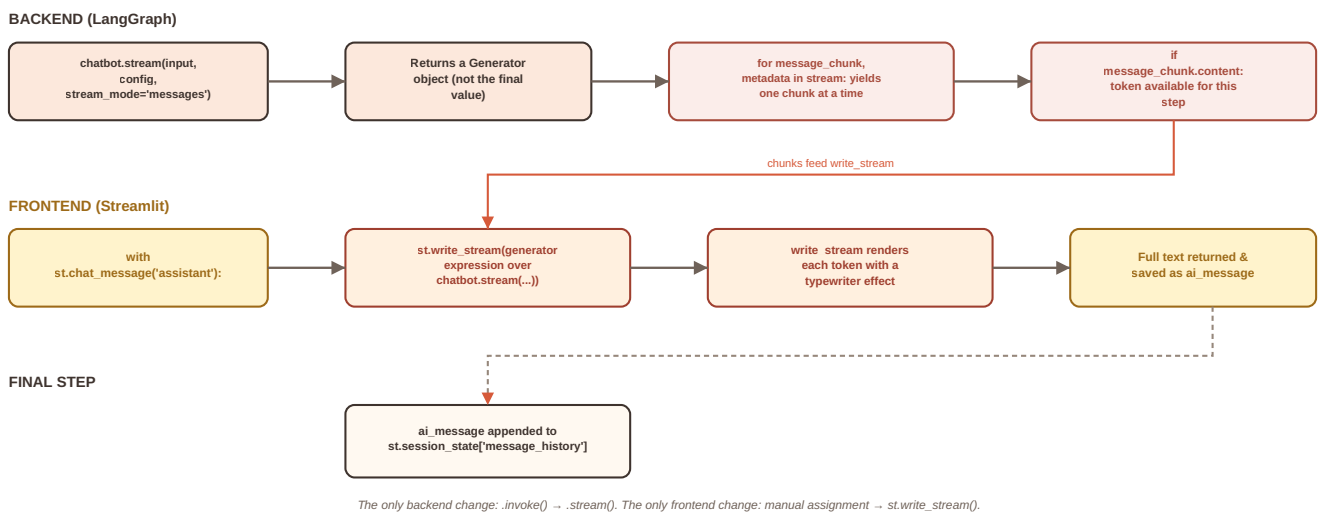


Figure 12.3 — End-to-end flow: `chatbot.stream()` yields `(message_chunk, metadata)` tuples in the backend; `st.write_stream` consumes a generator of `chunk.content` values and renders them with a typewriter effect in the frontend.

The Complete Frontend Code

The only part of the existing frontend that changes is the block that fetches and displays the assistant's response. The rest of the app — session state initialization, replaying message history, and the chat input box — stays exactly as it was in Chapter 11.

```

import streamlit as st
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage

# Define config
CONFIG = {'configurable':{'thread_id':'thread_1'}}

# st.session_state -> dict
if 'message_history' not in st.session_state:

    # Define message history in the format = [{'role':'...', 'content':'..'}]
    st.session_state['message_history'] = []

# Showing the whole message history before showing the current messages
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Input box
user_input = st.chat_input('Type here')

if user_input:

    # Add the message to message history first
    st.session_state['message_history'].append({'role':'user', 'content':user_input})

    with st.chat_message('user'):
        st.text(user_input)

    # Stream the AI response
    with st.chat_message('assistant'):
        ai_message = st.write_stream(
            message_chunk.content for message_chunk, metadata in chatbot.stream(
                {'messages': [HumanMessage(content = user_input)]},
                config = CONFIG,
                stream_mode = 'messages'
            )
        )

    # Add the message into message history
    st.session_state['message_history'].append({'role':'assistant', 'content':ai_message})

```

Reading the Key Block

The important construct is a **generator expression** passed directly into `st.write_stream`:

```
message_chunk.content for message_chunk, metadata in chatbot.stream(...)
```

This is a compact, inline generator that yields only `message_chunk.content` for every chunk produced by `chatbot.stream(...)`. `st.write_stream` consumes this generator directly: it iterates over it internally and renders each yielded piece of text with a typewriter effect, so no manual print loop or placeholder-updating code is required in the frontend.

Once streaming finishes, `st.write_stream` returns the **full concatenated text** of everything it streamed. That return value is captured in `ai_message`, which is then appended to `st.session_state['message_history']` exactly like a normal assistant turn — so streamed responses remain fully compatible with the existing message-history replay logic from Chapter 11.

IMPORTANT

`st.write_stream` does two jobs at once: it renders the typewriter UI effect *and* it returns the final assembled string. Skipping the assignment (`ai_message = st.write_stream(...)`) would still render the streaming text correctly on screen, but the response would never be saved into `message_history` — breaking conversation replay after a rerun.

COMMON MISTAKE

During the live build, the query passed to `chatbot.stream(...)` was accidentally hardcoded (e.g. always asking for a pasta recipe) instead of using the actual `user_input` variable from `st.chat_input`. This produced replies that had nothing to do with what the user actually typed. Always double-check that the live user input variable, not a placeholder string left over from backend testing, is what gets passed into the `HumanMessage` content.

🔄 Quick Revision — Frontend Implementation

- `st.write_stream` renders a generator to the app with a typewriter effect.
- Pass it a generator expression: (`message_chunk.content` for `message_chunk`, metadata in `chatbot.stream(...)`).
- `st.write_stream` returns the full concatenated streamed text once streaming completes.
- Capture that return value (e.g. `ai_message`) and append it to `st.session_state['message_history']`.
- Session state initialization, history replay, and `st.chat_input` remain unchanged from the non-streaming version.

K. Key Takeaways

- **Streaming** means an LLM sends tokens as soon as they are generated, instead of the caller waiting for the entire response before anything is returned.
- **.invoke() vs .stream()** — `.invoke()` blocks until the full result is ready; `.stream()` returns a generator that yields output incrementally as the graph executes.
- **Why it matters** — faster perceived response, more human-like/engaging interaction, better readability for long output (especially code), and the ability to stop early and save tokens/cost.
- **Python generators** use `yield` to produce values one at a time; `graph.stream(...)` is a generator, so it must be iterated (typically with a `for` loop) to receive output.

- **stream_mode='messages'** streams raw LLM token chunks; other modes (updates, values, custom) stream graph-execution state and become relevant for agentic status updates.
- **(message_chunk, metadata) tuples** are what stream_mode='messages' yields; message_chunk.content holds the actual streamed text.
- **Backend change** — the StateGraph, nodes, edges, and checkpointers are untouched; only chatbot.invoke(...) becomes chatbot.stream(..., stream_mode='messages').
- **Frontend change** — st.write_stream(generator) replaces a manual invoke-and-display block, rendering a typewriter effect and returning the full text to save into session state.

R. Revision Sheet

Streaming

Sending LLM output tokens as soon as each is generated, rather than waiting for the full response before returning anything.

stream_mode='messages'

Streaming mode that yields LLM token chunks as (message_chunk, metadata) tuples — used for typewriter-style text output.

.invoke()

Blocking call; returns only after the entire graph execution and full LLM response are complete.

message_chunk.content

The actual text of a single streamed token/chunk; should be checked for truthiness before use.

.stream()

Non-blocking-style call; immediately returns a generator that yields output incrementally as execution proceeds.

st.write_stream

Streamlit function that consumes a generator, renders it with a typewriter effect, and returns the full concatenated text once complete.

Generator (Python)

A special iterator that produces values one at a time using yield instead of return, pausing between each value.

st.status

Streamlit container for showing step-by-step status updates in agent-style applications (used in later, more agentic chapters).

I. Interview Questions

Q1. What does "streaming" mean in the context of LLM-based applications, and how does it differ from a standard blocking response?

Q2. What is the core difference between `chatbot.invoke(...)` and `chatbot.stream(...)` in LangGraph, in terms of what each call returns?

Q3. What Python language feature underlies LangGraph's `.stream()` method, and how does it differ from a normal function using `return`?

Q4. Name at least three distinct benefits of implementing streaming in a chatbot or agentic application.

Q5. What does `stream_mode='messages'` control, and how does it differ from `stream_mode='updates'` or `'values'`?

Q6. When iterating over a LangGraph stream with `stream_mode='messages'`, what do the two elements of each yielded tuple represent?

Q7. Why is it necessary to check `"if message_chunk.content:"` before printing or forwarding a chunk?

Q8. What does Streamlit's `st.write_stream` function do, and what does it return once streaming is complete?

Q9. Why must the return value of `st.write_stream` be captured and appended to session state, rather than relied upon only for its on-screen rendering effect?

Q10. How does streaming relate to token-based billing, and how can early stopping mid-response reduce cost?

F. Further Reading

- [LangGraph official documentation — Streaming guide](#) (`graph.stream()`, `stream_mode` options)
- [LangGraph official documentation — Streaming LLM tokens](#)
- [Streamlit official documentation — `st.write\_stream`](#)
- [Streamlit official documentation — Chat elements](#) (`st.chat_message`, `st.chat_input`, `st.status`)
- [Python official documentation — Generators and the `yield` statement](#)